

TD 2 : Techniques de Vectorisation du Texte

UPEC - EPISEN

Module : Traitement Automatique du Langage Naturel (NLP)

Objectif : passer d'un texte à des vecteurs numériques.

Objectifs du TD

À la fin de ce TD, l'étudiant sera capable de :

- Nettoyer et préparer un corpus textuel.
- Transformer un texte en vecteurs numériques.
- Comparer différentes méthodes de vectorisation.

1 Préparation de l'environnement

```
!pip install nltk scikit-learn pandas
```

2 Partie A : Préparation du corpus

2.1 Corpus de départ

Voici un petit corpus :

corpus = ["La transformation numérique accélère l'adoption de l'intelligence artificielle dans de nombreux secteurs économiques."

"De jeunes entreprises innovantes développent des solutions basées sur l'analyse de données massives."

"Plusieurs centres de recherche collaborent pour faire progresser les applications de la data science."

"Le traitement automatique du langage naturel permet d'automatiser la classification de documents."

"Les systèmes de dialogue intelligents améliorent la relation entre les organisations et leurs utilisateurs."

"Les infrastructures urbaines intègrent désormais des réseaux de capteurs pour optimiser les services publics."

"Le concept de tourisme intelligent mise sur la personnalisation de l'expérience grâce aux données et à l'IA."

"Les progrès en apprentissage automatique attirent de plus en plus de chercheurs et d'ingénieurs."]

2.2 Nettoyage et tokenisation

- Nettoyons le texte
- Mettre en minuscules
- Supprimer la ponctuation
- Retirer les stops words français

```

import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import string

nltk.download('punkt')
nltk.download('stopwords')
stop_words = set(stopwords.words('french'))

# Creer une nouvelle liste
corpus_clean = []

for text in corpus:
    tokens = word_tokenize(text.lower())
    tokens = [t for t in tokens if t.isalpha() and t not in
               stop_words]
    corpus_clean.append(" ".join(tokens))

print("Corpus_nettoyé:")
for i, doc in enumerate(corpus_clean):
    print(f"{i+1}. {doc}")

```

Questions

1. Quels types de mots ont été supprimés ?
2. Pourquoi est-il important de normaliser le texte avant la vectorisation ?
3. Combien de mots différents compte le corpus après nettoyage ?

3 Partie B : One-Hot Encoding

3.1 Principe

Chaque mot du vocabulaire est représenté par un vecteur binaire :

- 1 si le mot est présent dans le document ;
- 0 sinon.

3.2 Implémentation avec MultiLabelBinarizer

```

from sklearn.preprocessing import MultiLabelBinarizer

tokenized = [doc.split() for doc in corpus_clean]
mlb = MultiLabelBinarizer()
onehot = mlb.fit_transform(tokenized)
print("Vocabulaire:", mlb.classes_)
print("\nMatrice One-Hot:\n", onehot)

```

3.3 Questions

1. Que représente chaque ligne du tableau ?
2. Quelle est la taille du vecteur ?

4 Partie C : Bag of Words

4.1 Principe

La méthode *Bag of Words* (BoW) consiste à compter le nombre d'occurrences de chaque mot dans chaque document. Ainsi, contrairement au One-Hot Encoding, un mot peut apparaître plusieurs fois et sa fréquence est alors prise en compte.

4.2 Implémentation avec CountVectorizer

```
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer()
X_bow = vectorizer.fit_transform(corpus_clean)

print("Vocabulaire:", vectorizer.get_feature_names_out())
print("\nMatrice BoW:\n", X_bow.toarray())
```

4.3 Visualisation avec Pandas

```
import pandas as pd

df_bow = pd.DataFrame(
    X_bow.toarray(),
    columns=vectorizer.get_feature_names_out()
)
print(df_bow)
```

4.4 Questions

1. Quelle est la différence entre le One-Hot Encoding et le Bag of Words ?
2. Que se passe-t-il si un mot apparaît plusieurs fois dans un document ?
3. Quels mots semblent les plus fréquents dans le corpus ?

5 Partie D : TF-IDF

5.1 Principe

La méthode *TF-IDF* (Term Frequency – Inverse Document Frequency) pondère chaque mot selon deux critères :

- sa fréquence dans un document (TF) ;
- sa rareté dans le corpus (IDF).

Ainsi, un mot fréquent dans un document mais rare dans l'ensemble des documents recevra un poids élevé.

5.2 Implémentation avec TfidfVectorizer

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer()
X_tfidf = tfidf.fit_transform(corpus_clean)

print("Vocabulaire:", tfidf.get_feature_names_out())
print("\nMatrice TF-IDF:\n", X_tfidf.toarray())
```

5.3 Visualisation avec Pandas

```
import pandas as pd

df_tfidf = pd.DataFrame(
    X_tfidf.toarray(),
    columns=tfidf.get_feature_names_out()
)

print(df_tfidf)
```

5.4 Questions

1. Quelle différence remarquez-vous entre les valeurs de TF-IDF et celles du Bag of Words?
2. Quels mots ont le poids le plus élevé? Pourquoi?
3. Pourquoi TF-IDF est-il souvent préféré au Bag of Words dans les systèmes de recherche d'information?